



Despliegue de aplicaciones en contenedores DAC

UD4 Despliegue de aplicaciones con Docker

Ejercicios obligatorios.....	2
1 y 2) Configura docker-compose.yml para poder desplegar un servicio de MySQL junto con phpMyAdmin.....	2
3) Instalación de PrestaShop.....	6
4) Visualización de la base de datos.....	9
Ejercicios opcionales.....	11
Test de autoevaluación.....	11
Testear cómo funciona el aislamiento de redes con Docker.....	11
Ejemplos de networking.....	13
Bibliografía web.....	16
Valoración personal.....	17



Ejercicios obligatorios

1 y 2) Configura docker-compose.yml para poder desplegar un servicio de MySQL junto con phpMyAdmin

Procederemos a generar un YAML que representa un docker-compose para montar estos servicios, integrarlos y exponerlos.

Se ha desarrollado este YAML, se definen los dos servicios con sus redirecciones y variables, así como la persistencia para mysql y una red que compartirán:

```
version: '3.8'

services:
  db_mysql:
    image: mysql:8.0 #Ponemos versión a capón de la bbdd, nos aseguramos que por este lado será estable
    container_name: mysql_db
    restart: always # Se reinicia solo a no ser que lo tiremos nosotros
    environment:
      MYSQL_ROOT_PASSWORD: admin
      MYSQL_DATABASE: php_db
      MYSQL_USER: usuario
      MYSQL_PASSWORD: admin
    volumes:
      - db_data:/var/lib/mysql
    networks:
      - app_network

  # Servicio para phpMyAdmin
  phpmyadmin:
    image: phpmyadmin:latest # la última
    container_name: phpmyadmin
    restart: always
    ports:
      - "8091:80"
    environment:
      PMA_HOST: db_mysql
      PMA_PORT: 3306 # Puerto por defecto de MySQL
      MYSQL_ROOT_PASSWORD: admin
    networks:
      - app_network # Misma red
    depends_on:
      # Asegura que el servicio db_mysql se inicia antes
      - db_mysql

volumes:
  db_data:
    driver: local

networks:
```



Juan Cruz Garrido Suso

Práctica Unidad 4

Despliegue de aplicaciones en contenedores

```
app_network:  
  driver: bridge
```

Y se crea con un nano en una carpeta creada para este ejercicio:

```
version: '3.8'  
  
services:  
  db_mysql:  
    image: mysql:8.0 #POnemos versión a capón de la bbdd, nos aseguramos que por este lado será estable  
    container_name: mysql_db  
    restart: always # Se reinicia solo a no ser que lo tiremos nosotros  
    environment:  
      MYSQL_ROOT_PASSWORD: admin  
      MYSQL_DATABASE: php_db  
      MYSQL_USER: usuario  
      MYSQL_PASSWORD: admin  
    volumes:  
      - db_data:/var/lib/mysql  
    networks:  
      - app_network  
  
  # Servicio para phpMyAdmin  
  phpmyadmin:  
    image: phpmyadmin:latest # la última  
    container_name: phpmyadmin  
    restart: always  
    ports:  
      - "8091:80"  
    environment:  
      PMA_HOST: db_mysql  
      PMA_PORT: 3306 # Puerto por defecto de MySQL  
      MYSQL_ROOT_PASSWORD: admin  
    networks:  
      - app_network # Misma red  
    depends_on:  
      # Asegura que el servicio db_mysql se inicia antes  
      - db_mysql  
  
volumes:  
  db_data:  
    driver: local  
  
networks:  
  app_network:  
    driver: bridge  
delarioja@ubuntu-juancruz:~/ejercicio_ud_4$ |
```

nano y cat de nuestro nuevo docker-compose.yml

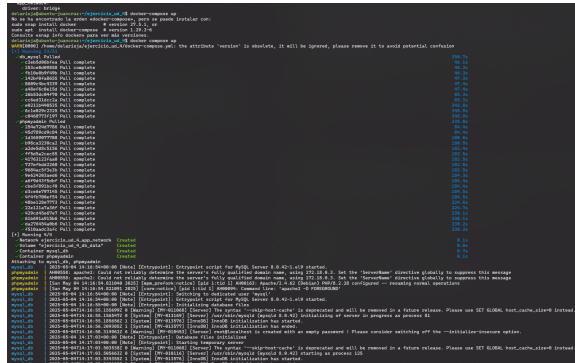
Tras esto, lanzamos el comando `docker compose up` en ese mismo directorio.

NOTA: Podríamos haber cambiado de nombre al yaml y llamarlo en cada caso con `-f nombre_del_yaml.yml`, pero en este caso usamos el nombre por defecto.

Juan Cruz Garrido Suso

Práctica Unidad 4

Despliegue de aplicaciones en contenedores



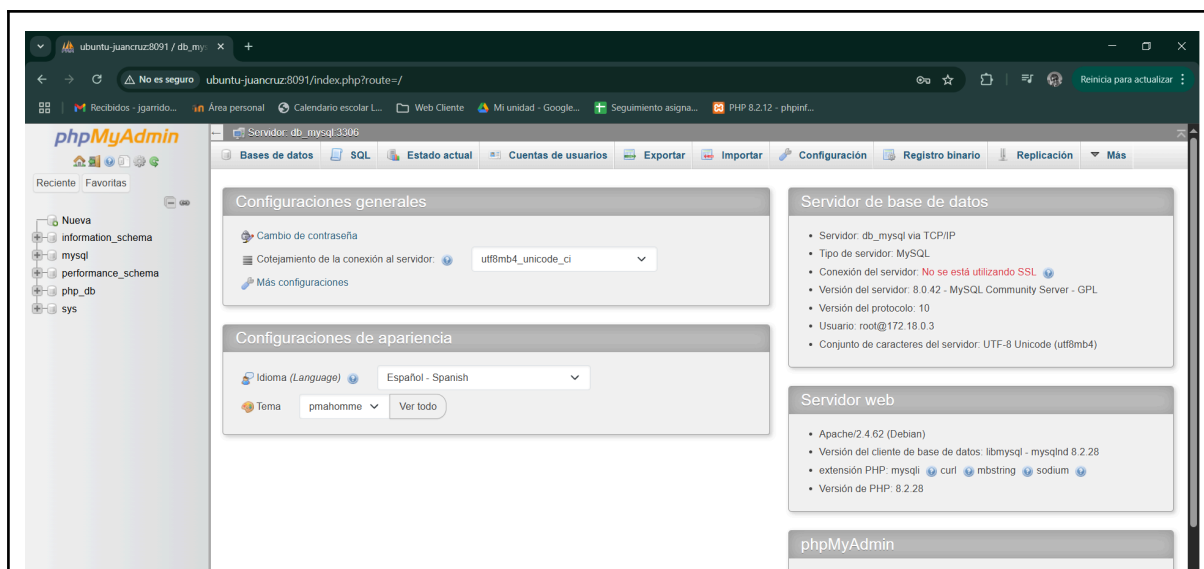
```
docker compose up
```

Diferentes partes del proceso de lanzamiento, en primer lugar hace el pull de las imágenes requeridas y posteriormente lanza los procesos.

Deja la consola ocupada con las trazas que va generando.

Podríamos haber usado la opción `-d` que se refiere a modo detached o separado para evitar que el proceso docker se “quede” con la consola, pero de este modo en esta terminal podremos observar el comportamiento de los procesos de los contenedores.

Ahora comprobaremos que nuestro phpmyadmin de la bbdd está levantado:



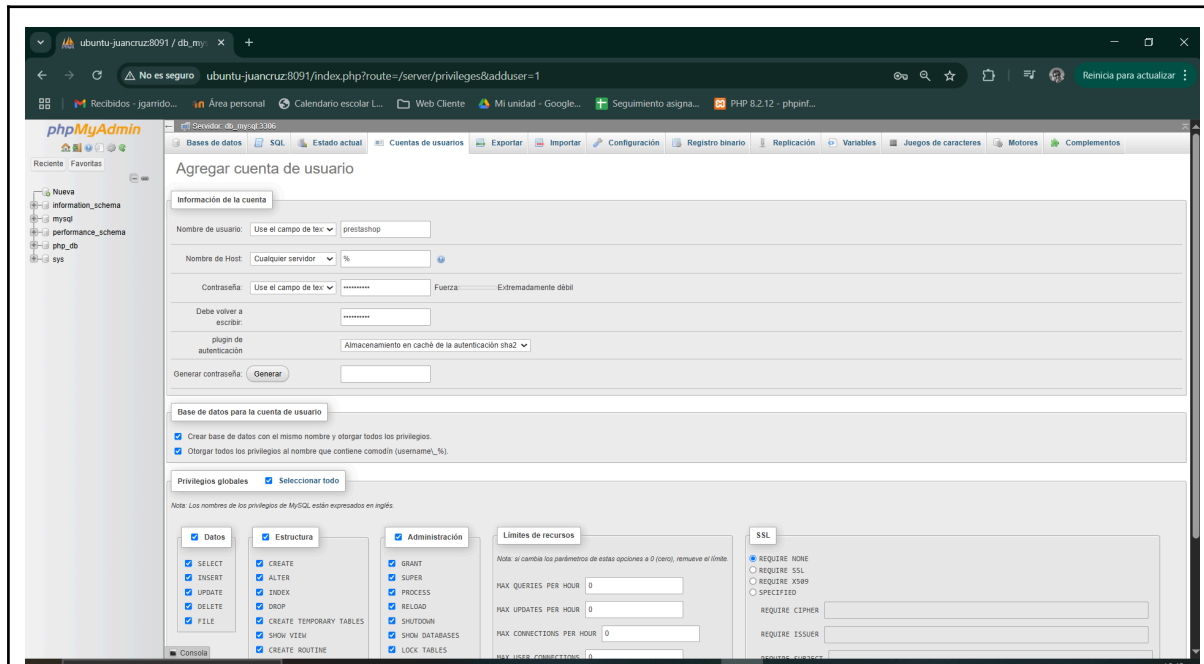
Y listos, tras logarnos como root:admin hemos accedido desde el host.

Ahora crearemos el usuario prestashop y una base de datos asociada de mismo nombre, como se nos solicita.

Juan Cruz Garrido Suso

Práctica Unidad 4

Despliegue de aplicaciones en contenedores



Pantalla de creación del nuevo usuario

Ahora se nos pide que tiremos este compose y comprobemos que docker así nos lo indica, para ello, en una nueva terminal:

```
delarioja@ubuntu-juancruz:~/ejercicio_ud_4$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
d4b464797e44   phpmyadmin:latest                  "/docker-entrypoint..." 6 minutes ago  Up 6 minutes  0.0.0.0:8091->80/tcp, [::]:8091->80/tcp
425d21821091   mysql:8.0                          "docker-entrypoint.s..." 27 minutes ago Up 6 minutes  3306/tcp, 33060/tcp
56a8b0f9eb6a   portainer/portainer-ce:latest     "/portainer"             3 hours ago   Up 3 hours    0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:9443->9443/tcp, [::]:9443->9443/tcp, 9080/tcp
delarioja@ubuntu-juancruz:~/ejercicio_ud_4$ docker compose down
WARN[0000] /home/delarioja/ejercicio_ud_4/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 3/3
  ✓ Container phpmyadmin      Removed          1.3s
  ✓ Container mysql_db        Removed          1.6s
  ✓ Network ejercicio_ud_4_app_network Removed          0.2s
delarioja@ubuntu-juancruz:~/ejercicio_ud_4$ docker ps
CONTAINER ID   IMAGE                                NAMES                  CREATED        STATUS        PORTS
56a8b0f9eb6a   portainer/portainer-ce:latest     "/portainer"             3 hours ago   Up 3 hours    0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:9443->9443/tcp, [::]:9443->9443/tcp, 9080/tcp
delarioja@ubuntu-juancruz:~/ejercicio_ud_4$
```

```
docker ps
docker compose down
docker ps
```

Vemos como el primer ps nos indica que los servicios están levantados y que tras tirarlos, el último nos informa correctamente de que ya no.

NOTA: En la otra terminal el proceso también ha finalizado:



Juan Cruz Garrido Suso

Práctica Unidad 4

Despliegue de aplicaciones en contenedores

```
537.36 (KHTML, like Gecko) Chrome/135.0.0.0 Safari/537.36
phpmyadmin | [Sun May 04 14:45:04.137978 2025] [mpm_prefork:notice] [pid 1:tid 1] AH00170: caught SIGWINCH, shutting down gracefully
phpmyadmin exited with code 0
mysql_db | 2025-05-04T14:45:05.426751Z 0 [System] [MY-013172] [Server] Received SHUTDOWN from user <via user signal>. Shutting down mysqld (Version: 8.0.42).
mysql_db | 2025-05-04T14:45:06.716659Z 0 [System] [MY-010918] [Server] /usr/sbin/mysqld: Shutdown complete (mysqld 8.0.42) MySQL Community Server - GPL.
mysql_db exited with code 0
delarioja@ubuntu-juancruz:~/ejercicio_ud_4$ |
Enable Watch
```

3) Instalación de PrestaShop

Revisamos nuestro docker-compose para incluir el compose recomendado por los desarrolladores, usando el servicio y red mysql previamente definido:

```
version: '3.8'

services:
  db_mysql:
    image: mysql:8.0 #Ponemos versión a capón de la bbdd, nos aseguramos que por este lado será estable
    container_name: mysql_db
    restart: always # Se reinicia solo a no ser que lo tiremos nosotros
    environment:
      MYSQL_ROOT_PASSWORD: admin
      MYSQL_DATABASE: php_db
      MYSQL_USER: usuario
      MYSQL_PASSWORD: admin
    volumes:
      - db_data:/var/lib/mysql
    networks:
      - app_network

  # Servicio para phpMyAdmin
  phpmyadmin:
    image: phpmyadmin:latest # la última
    container_name: phpmyadmin
    restart: always
    ports:
      - "8080:80"
    environment:
      PMA_HOST: db_mysql
      PMA_PORT: 3306 # Puerto por defecto de MySQL
      MYSQL_ROOT_PASSWORD: admin
    networks:
      - app_network # Misma red
    depends_on:
      # Asegura que el servicio db_mysql se inicia antes
      - db_mysql

  prestashop:
    container_name: prestashop
    image: prestashop/prestashop:latest
    restart: unless-stopped #Se reinicia solo a no ser que lo hayamos tirado, y se mantiene tiorado aunque
    reiniciemos el docker hasta que lo levantemos manualmete
    depends_on:
      - db_mysql
    ports:
      - 80:80
    environment:
      DB_SERVER: db_mysql
      DB_NAME: prestashop
      DB_USER: root
      DB_PASSWD: admin
    networks:
      - app_network # Misma red
```




Juan Cruz Garrido Suso

Práctica Unidad 4

Despliegue de aplicaciones en contenedores

```
volumes:
  db_data:
    driver: local

networks:
  app_network:
    driver: bridge
```

Tras editar lanzamos el composer.

```
delarioja@ubuntu-juancruz:/ejercicio_ud_4$ sudo nano docker-compose.yml
[sudo] contraseña para delarioja:
delarioja@ubuntu-juancruz:/ejercicio_ud_4$ docker compose up
[0000] /home/delarioja/ejercicio_ud_4/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 4/4
  prestashop Pulled
    7c6326d51a Pull complete
    41bfba57aa2a Pull complete
    7f653322295 Pull complete
    90a93969485 Pull complete
    ccca7c18c4b Pull complete
    a098ac43b7b Pull complete
    79965c5b56e Pull complete
    5f6233b2d6e Pull complete
    823862a769b Pull complete
    0b4b7f8b4f6 Pull complete
    012fd93ee07a Pull complete
    a3339e6f2db1 Pull complete
    83a033794cb Pull complete
    4f4f6788ef5d Pull complete
    23a4e4d91ba Pull complete
    3bac4b93b1c Pull complete
    5e8338e4276 Pull complete
    617a6e34f227 Pull complete
    ab5f6388c497 Pull complete
    55a4fc0b118 Pull complete
    6e897bc9fad7 Pull complete
    0b2a1a7e937b Pull complete
    fd2d653e9876 Pull complete
    9c2a88b8e6d8 Pull complete
    95a2eadc4bc8 Pull complete
    d75bee08e26 Pull complete
    1f6a1ff94ce Pull complete
[+] Running 4/4
  Network ejercicio_ud_4_app_network Created
  Container mysql_db Created
  Container prestashop Created
  Container phpmyadmin Created
Attaching to mysql_db, phpmyadmin, prestashop
mysql_db_1 2025-05-04 15:17:27+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.42-1.el9 started.
mysql_db_1 2025-05-04 15:17:27+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
prestashop_1 Setting up install lock file...
prestashop_1 Reapplying PrestaShop files for enabled volumes ...
prestashop_1 Copying files from tmp directory ...
mysql_db_1 2025-05-04 15:17:27+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.42-1.el9 started.
phpmyadmin_1 AM080508: apache2: Could not reliably determine the server's fully qualified domain name, using 172.18.0.4. Set the 'ServerName' directive globally to suppress this message
phpmyadmin_1 AM080508: apache2: Could not reliably determine the server's fully qualified domain name, using 172.18.0.4. Set the 'ServerName' directive globally to suppress this message
phpmyadmin_1 [Sun May 04 15:17:27 2025] [warn] [pid 1:tid 1] AH00163: Apache/2.4.62 (Debian) PHP/8.2.28 configured -- resuming normal operations
phpmyadmin_1 [Sun May 04 15:17:27 2025] [core:notice] [pid 1:tid 1] AH00954: Command line: 'apache2 -D FPM-CM0000'
mysql_db_1 'var/lib/mysql/mysql.sock' -> '/var/run/mysqld/mysqld.sock'
mysql_db_1 2025-05-04 15:17:28.042839Z 0 [Warning] [MY-011065] [Server] The syntax '--skip-host-cache' is deprecated and will be removed in a future release. Please use SET GLOBAL host_cache_size=0 instead.
mysql_db_1 2025-05-04 15:17:28.045753Z 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 8.0.42) starting as process 1
mysql_db_1 2025-05-04 15:17:28.083831Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
```

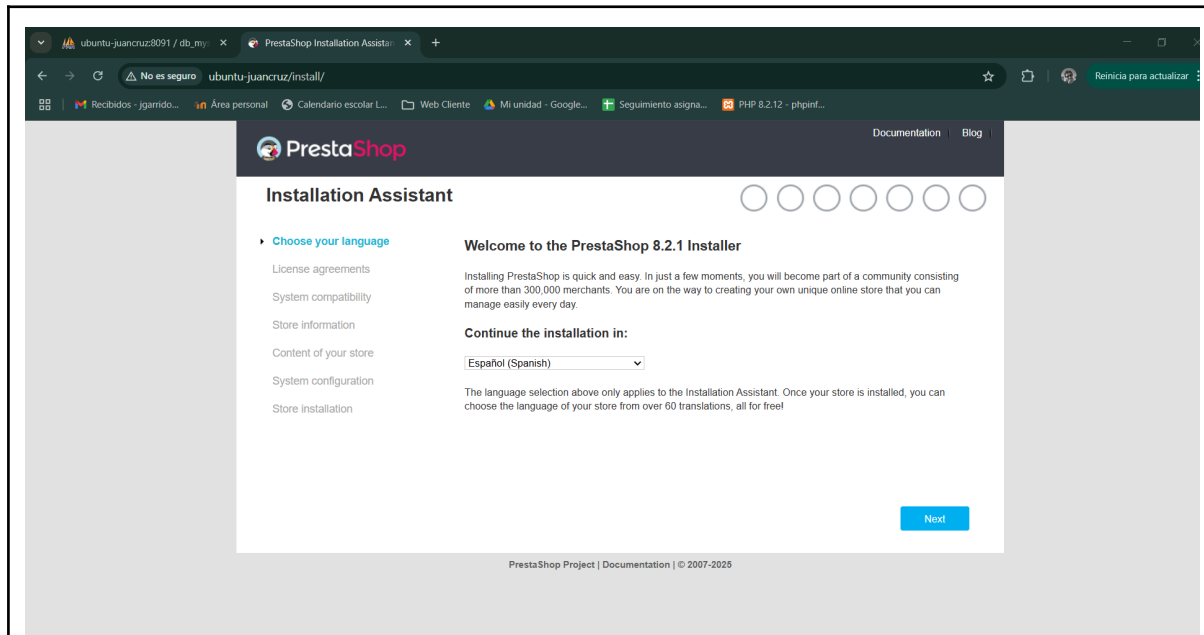
Edición con nano y lanzamiento con docker compose up

NOTA: A pesar de que en los contenidos de la lección se usa `version`, docker se me está quejando por hacerlo, al parecer está obsoleto, se conoce que confían en que no habrá más versiones de la sintaxis.

Juan Cruz Garrido Suso

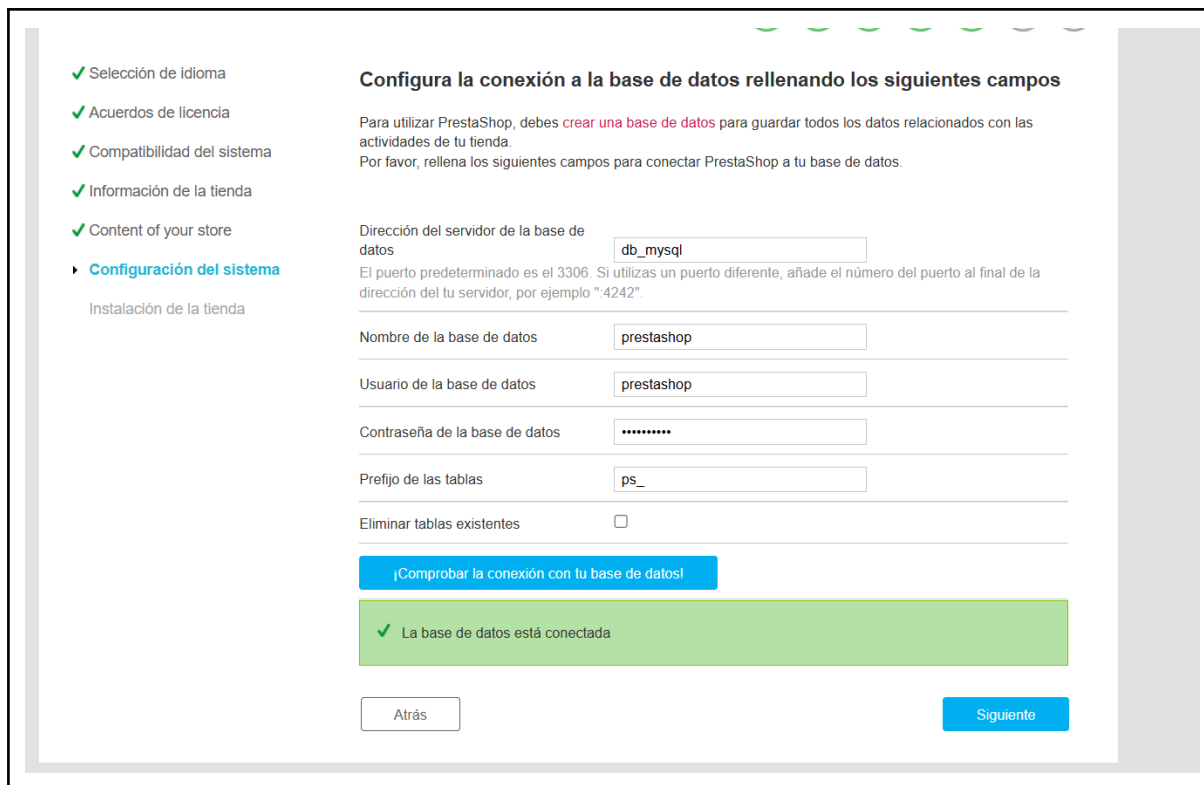
Práctica Unidad 4

Despliegue de aplicaciones en contenedores



Y en efecto, comprobamos que accedemos por el puerto 80, como está configurado.

El paso siguiente consiste en instalar a través de la propia web del servicio:



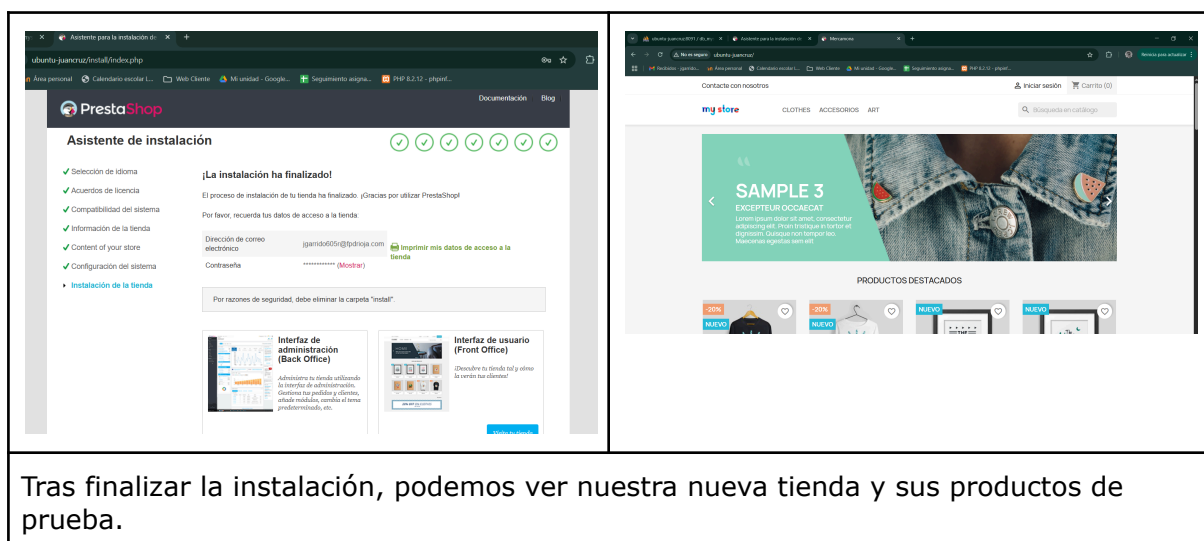
Juan Cruz Garrido Suso

Práctica Unidad 4

Despliegue de aplicaciones en contenedores

Comprobamos que en efecto tenemos acceso a mysql, la dirección del servicio es el nombre dado en el composer.

NOTA: En este punto ya comprendo que, de hecho, la bbdd y el usuario para esta aplicación podrían haberse creado directamente desde el composer al definir `db_mysql`, ahorrándonos la generación de la tabla desde phpmyadmin. En el composer usado había puesto yo `php_db` sin entender aún el propósito del ejercicio.



The left screenshot shows the PrestaShop installation assistant. It lists steps: Selection of language, License agreement, System compatibility, Store information, Content of your store, and System configuration. The installation is marked as complete. It provides the email `jjgarridos@larioja.com` and a password field. It also shows thumbnails for the 'Interfaz de administración (Back Office)' and 'Interfaz de usuario (Front Office)'. The right screenshot shows the 'my store' storefront. It features a header with navigation links (CLOTHES, ACCESORIOS, ART), a search bar, and a main banner for 'SAMPLE 3' with a denim jacket. Below the banner are 'PRODUCTOS DESTACADOS' (Featured Products) with 'NUEVO' (New) tags.

Tras finalizar la instalación, podemos ver nuestra nueva tienda y sus productos de prueba.

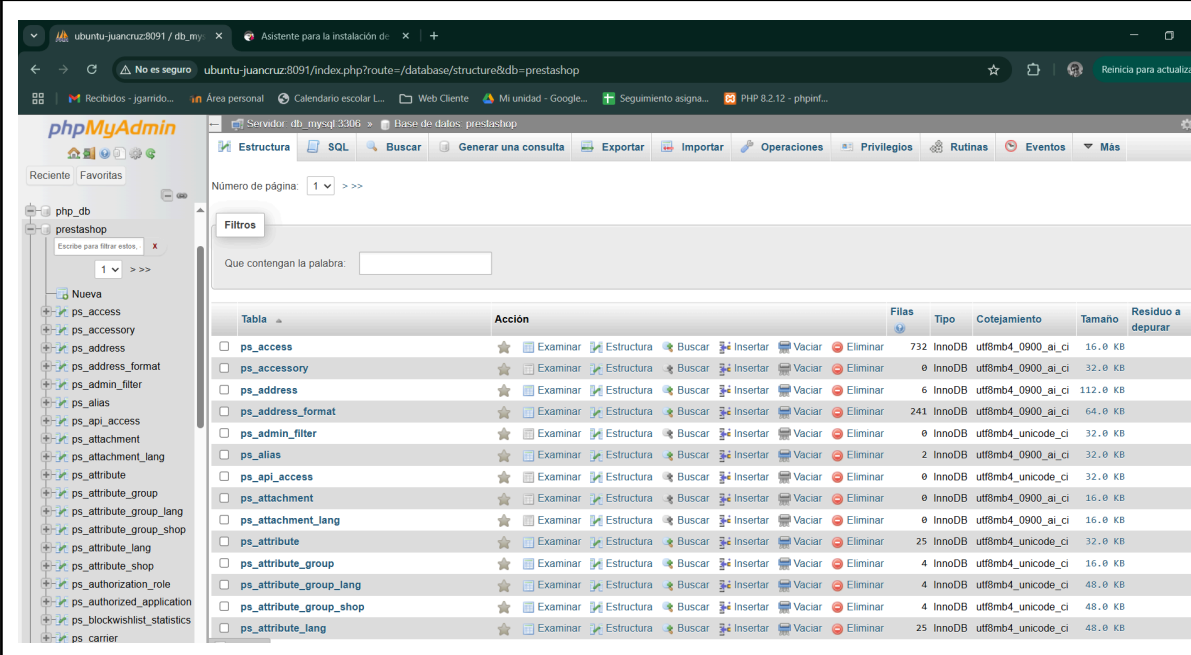
4) Visualización de la base de datos

Tras finalizar el proceso de instalación, comprobamos que las tablas se han creado:

Juan Cruz Garrido Suso

Práctica Unidad 4

Despliegue de aplicaciones en contenedores



Visualización de las nuevas tablas en la bbdd prestashop a través de phpmyadmin.

Como colofón, resaltar de nuevo que la bbdd prestashop creo que debería haberse creado en el propio composer sin necesitar ir a phpmyadmin para hacerlo, eso le daría robustez.

Por otro lado, las credenciales inyectadas al servicio prestashop en forma de variables de entorno no son usadas ya que en el proceso de instalación nos las vuelve a pedir. Consultando esto, al parecer conviene ponerlas de todos modos en el composer como una forma de documentar, ya que otros contenedores (como el de mysql) sí está configurado para leer esas variables y generar así bbdd y usuarios al levantar el composer. No he comprobado si todo funciona si las quito.

Juan Cruz Garrido Suso

Práctica Unidad 4

Despliegue de aplicaciones en contenedores

Ejercicios opcionales

Test de autoevaluación

Comenzado el	domingo, 4 de mayo de 2025, 15:15
Estado	Finalizado
Finalizado en	domingo, 4 de mayo de 2025, 15:22
Tiempo empleado	6 minutos 48 segundos
Calificación	10,00 de 10,00 (100%)

Pregunta 1
Correcta
Se puntúa 1,00

El principal objetivo de Docker Compose es:
Seleccione una:

Navegación por el cuestionario

1	2	3	4	5	6	7	8	9
✓	✓	✓	✓	✓	✓	✓	✓	✓
10								
✓								

[Mostrar una página cada vez](#)
[Finalizar revisión](#)

Testear cómo funciona el aislamiento de redes con Docker

Creamos dos redes y comprobamos que están ahí:

```
delarioja@ubuntu-juancruz:~/ejercicio_ud_4$ docker network create network1
0664e2fe460257001f1d0c3fff173aeffcc982d1b0c26ee7bc4464c3bb291d97
delarioja@ubuntu-juancruz:~/ejercicio_ud_4$ docker network create network2
0061d0c6041bd1e40f3b0c005e8aca2dce0d767da4c37cc21c59f5cceda9145b
delarioja@ubuntu-juancruz:~/ejercicio_ud_4$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
ca78bf32b20f        bridge             bridge             local
4252c4155e1c        host               host               local
0664e2fe4602        network1          bridge            local
0061d0c6041b        network2          bridge            local
7fb3e82dafb8        none              null               local
delarioja@ubuntu-juancruz:~/ejercicio_ud_4$
```

```
docker network create network1
docker network create network2
docker network ls
```

El tipo bridge es el tipo de red por defecto.

Ahora vamos a levantar dos contenedores sobre la primera red y comprobar que se ven:



Juan Cruz Garrido Suso

Práctica Unidad 4

Despliegue de aplicaciones en contenedores

```
delarioja@ubuntu-juancruz:~/ejercicio_ud_4$ docker run -it --rm --name contenedor_ud_4 --network network1 alpine sh
/ # ls
bin    dev    etc    home  lib    media  mnt    opt    proc   root   run    sbin   srv    sys    tmp    usr    var
/ # exit
delarioja@ubuntu-juancruz:~/ejercicio_ud_4$ docker run -dit --rm --name contenedor_ud_4 --network network1 alpine sh
e3d92c7048853861e96273945b61159ee2eac4e64159ed341ea7220816cbfb76
delarioja@ubuntu-juancruz:~/ejercicio_ud_4$ docker run -it --rm --name contenedor_ud_4_2 --network network1 alpine sh
/ # ping contenedor_ud_4
PING contenedor_ud_4 (172.18.0.2): 56 data bytes
64 bytes from 172.18.0.2: seq=0 ttl=64 time=0.091 ms
64 bytes from 172.18.0.2: seq=1 ttl=64 time=0.061 ms
64 bytes from 172.18.0.2: seq=2 ttl=64 time=0.078 ms
64 bytes from 172.18.0.2: seq=3 ttl=64 time=0.067 ms
64 bytes from 172.18.0.2: seq=4 ttl=64 time=0.089 ms
64 bytes from 172.18.0.2: seq=5 ttl=64 time=0.081 ms
64 bytes from 172.18.0.2: seq=6 ttl=64 time=0.078 ms
64 bytes from 172.18.0.2: seq=7 ttl=64 time=0.098 ms
64 bytes from 172.18.0.2: seq=8 ttl=64 time=0.077 ms
64 bytes from 172.18.0.2: seq=9 ttl=64 time=0.084 ms
64 bytes from 172.18.0.2: seq=10 ttl=64 time=0.073 ms
^C
--- contenedor_ud_4 ping statistics ---
11 packets transmitted, 11 packets received, 0% packet loss
round-trip min/avg/max = 0.061/0.079/0.098 ms
/ #
```

```
docker run -dit --rm --name contenedor_ud_4 --network network1 alpine sh
```

Con esa instrucción lanzamos el primer contenedor, detached, con linea de comandos por si lo requeriésemos, con opción de autoborrado al terminar, con el nombre contenedor_ud_4 , con la red network1, con un alpine y con posibilidad de sh.

```
docker run -it --rm --name contenedor_ud_4_2 --network network1 alpine sh
```

Con esto ejecutamos otro contenedor en la misma red, con nombre contenedor_ud_4_2 y en modo no detached, de modo que nos aparece la consola y en ella hacemos un ping al contenedor anterior con éxito, en efecto, están los dos levantados y en la misma red.

NOTA: Algunos comandos útiles para este trabajo han sido:

docker exec -it contenedor_ud_4 sh (ejecutará un sh con pantalla y teclado sobre un contenedor en marcha)

docker compose exec db_mysql sh (CON composer, super)

docker container rm contenedor_ud_4

docker container stop contenedor_ud_4

docker container ls

docker ps -a

Ahora que hemos comprobado la conectividad entre los dos, haremos una prueba en la que levantamos dos uno en cada red, y veremos como en ese caso no se verán.

Juan Cruz Garrido Suso

Práctica Unidad 4

Despliegue de aplicaciones en contenedores

```
delarioja@ubuntu-juancruz: ~/ X delarioja@ubuntu-juancruz: ~ X + v
delarioja@ubuntu-juancruz:~$ docker run -dit --rm --name contenedor_ud_4 --network network1 alpine sh
acd0331ca0d605c73fccee533220d373a6346bcb21bf0ecccalf6e82bd218921d
delarioja@ubuntu-juancruz:~$ docker run -it --rm --name contenedor_ud_4_2 --network network2 alpine sh
/ # ping contenedor_ud_4_2
PING contenedor_ud_4_2 (172.19.0.2): 56 data bytes
64 bytes from 172.19.0.2: seq=0 ttl=64 time=0.045 ms
64 bytes from 172.19.0.2: seq=1 ttl=64 time=0.057 ms
^C
--- contenedor_ud_4_2 ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 0.045/0.051/0.057 ms
/ # ping contenedor_ud_4
ping: bad address 'contenedor_ud_4'
/ #
```

Aquí hemos comprobado que en efecto, al estar en diferentes redes, el segundo no ve al primero.

Ejemplos de networking

En el video del Pelado Nerd se configuran dos contenedores con driver `macvlan`, lo que los monta como si fueran máquinas en la red local, que ven todo y son vistos por todos en esa red.

Para ejemplificar el caso se me ocurre montar un apache configurado así y tras averiguar los detalles de mi red, configurarlo, lanzarlo y consultarlo desde mi host en esa ip.

```
delarioja@ubuntu-juancruz:~/ejercicio_ud_4$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:48:6c:d3 brd ff:ff:ff:ff:ff:ff
    inet 192.168.30.182/24 brd 192.168.30.255 scope global dynamic noprefixroute enp0s3
        valid_lft 667sec preferred_lft 667sec
    inet6 2a02:9130:fe0c:bf87:f903:6a02:365d:5566/64 scope global temporary dynamic
        valid_lft 4269sec preferred_lft 4269sec
    inet6 2a02:9130:fe0c:bf87:a00:27ff:fe48:6cd3/64 scope global dynamic mngtmpaddr
        valid_lft 4269sec preferred_lft 4269sec
    inet6 fe80::a00:27ff:fe48:6cd3/64 scope link
        valid_lft forever preferred_lft forever
```

`ip a`

Nos da los detalles de nuestra red, vemos que `enp0s3` en nuestra interfaz y nuestra máscara de subred `192.168.30.0/24`

Ahora generamos un yaml sencillo para levantar un apache:

```
version: '3.9'
```




Juan Cruz Garrido Suso

Práctica Unidad 4

Despliegue de aplicaciones en contenedores

```
services:
  apache_macvlan:
    image: httpd:latest
    container_name: mi_apache_macvlan
    restart: unless-stopped
    networks:
      lan_macvlan:
        ipv4_address: 192.168.30.90

networks:
  lan_macvlan:
    driver: macvlan
    driver_opts:
      parent: enp0s3
    ipam:
      config:
        - subnet: 192.168.30.0/24
          gateway: 192.168.30.44
```

```
delarioja@ubuntu-juancruz:~/ejercicio_ud_4$ mkdir pelaonerd
delarioja@ubuntu-juancruz:~/ejercicio_ud_4$ cd pelaonerd/
delarioja@ubuntu-juancruz:~/ejercicio_ud_4/pelaonerd$ sudo nano compose.yaml
[sudo] contraseña para delarioja:
delarioja@ubuntu-juancruz:~/ejercicio_ud_4/pelaonerd$ cat compose.yaml
version: '3.9'

services:
  apache_macvlan:
    image: httpd:latest
    container_name: mi_apache_macvlan
    restart: unless-stopped
    networks:
      lan_macvlan:
        ipv4_address: 192.168.30.90

networks:
  lan_macvlan:
    driver: macvlan
    driver_opts:
      parent: enp0s3
    ipam:
      config:
        - subnet: 192.168.30.0/24
          gateway: 192.168.30.44
delarioja@ubuntu-juancruz:~/ejercicio_ud_4/pelaonerd$ |
```

Creamos un directorio para el ejercicio y un yaml con un apache, configurado con macvlan. Busco y compruebo que podríamos usar DHCP, pero no lo hago así para que la ip del servicio la conozca de antemano.

Lanzamos el servicio:



Juan Cruz Garrido Suso

Práctica Unidad 4

Despliegue de aplicaciones en contenedores

```
delarioja@ubuntu-juancruz:~/ejercicio_ud_4/pelaonerd$ docker compose up
WARN[0000] /home/delarioja/ejercicio_ud_4/pelaonerd/compose.yaml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[*] Running 5/7
+ apache_macvlan [#####] 30.23MB / 30.26MB Pulling
  254e724d7786 Already exists
  18d91782dc62 Pull complete
  4f4fb700ef54 Pull complete
  4ceee7b3d76 Pull complete
  0ff470512d2f Extracting [>] 262.1kB/26.06MB
  ba78a05e3b3c Download complete
```

docker compose up

Levantando el servicio

```
delarioja@ubuntu-juancruz:~/ejercicio_ud_4/pelaonerd$ docker compose up
WARN[0000] /home/delarioja/ejercicio_ud_4/pelaonerd/compose.yaml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[*] Running 1/1
  Container mi_apache_macvlan Created
Attaching to mi_apache_macvlan
mi_apache_macvlan | AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 192.168.30.90. Set the 'ServerName' directive globally to suppress this message
mi_apache_macvlan | AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 192.168.30.90. Set the 'ServerName' directive globally to suppress this message
mi_apache_macvlan | [Sun May 04 21:02:49.047952 2025] [mpm_event:notice] [pid 1:tid 1] AH00489: Apache/2.4.63 (Unix) configured -- resuming normal operations
mi_apache_macvlan | [Sun May 04 21:02:49.048146 2025] [core:notice] [pid 1:tid 1] AH00094: Command line: 'httpd -D FOREGROUND'
```

Sin embargo no funciona, ignoro la razón, la red parece estar disponible y siendo usada por el contenedor:



Juan Cruz Garrido Suso

Práctica Unidad 4

Despliegue de aplicaciones en contenedores

```
delarioja@ubuntu-juancruz:~/ejercicio_ud_4/pelaonerd$ docker network inspect pelaonerd_lan_macvlan
[
  {
    "Name": "pelaonerd_lan_macvlan",
    "Id": "8736dac377c0397ad05d6d3674d339b86e20d546dde48bad75ee1de3ff6d28f6",
    "Created": "2025-05-04T22:42:30.03453461+02:00",
    "Scope": "local",
    "Driver": "macvlan",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": [
        {
          "Subnet": "192.168.30.0/24",
          "Gateway": "192.168.30.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Containers": {},
    "Options": {
      "parent": "enp0s3"
    },
    "Labels": {
      "com.docker.compose.config-hash": "13f7888e2ded34572916c7c7789cdf6c372d58d23cfa8a65597494eb66a120f4",
      "com.docker.compose.network": "lan_macvlan",
      "com.docker.compose.project": "pelaonerd",
      "com.docker.compose.version": "2.35.1"
    }
  }
]
```

Aún así no logro desentrañar el problema y descarto firewall por que las pruebas anteriores han funcionado. Al parecer por wifi este driver puede dar algunas veces problemas, o quizá sea que estoy usando una red montada desde el celular por que no estoy en mi domicilio habitual.

En cualquier caso lo dejo así, quizá haya tiempo de que podamos resolverlo, podríamos probar por DHCP.

Y creo que con esto he finalizado la tarea.

Bibliografía web

- <https://docs.portainer.io/start/install-ce/server/docker/linux>
- <https://docs.docker.com/engine/install/ubuntu/>
- <https://claude.ai/>
- <https://aistudio.google.com>



Juan Cruz Garrido Suso

Práctica Unidad 4

Despliegue de aplicaciones en contenedores

- <https://devdocs.prestashop-project.org/8/basics/installation/environments/docker/>
- <https://www.datacamp.com/tutorial/set-up-and-configure-mysql-in-docker>
- <https://www.docker.com/blog/how-to-use-the-apache-httpd-docker-official-image>

Valoración personal

Realmente ha sido de lo más esclarecedora esta asignatura al respecto de porqué usar docker y los contenedores. La facilidad para montar fácilmente entornos complejos, con imágenes ya definidas con todas sus dependencias, con las variables de entorno fáciles de establecer, conectividad explícita y sencilla y con persistencia a mano.

He visto algo en algún video del amigo Pelado Nerd, en el que se usan contenedores para compilar y empaquetar y por otro lado contenedores muy pequeños para servir la aplicación. Me parece muy útil y muy práctico, y lo que nos queda por aprender. De haberlo sabido no tendría un XAMPP instalado en mi windows.

Por lo demás un verdadero placer haber sido tu alumno en este mi primer año, espero que todo vaya bien y te deseo mucha suerte y felicidad.